

EECS 498 · AASE · FALL 2026

Hackathon 1, and Hackathon 1.5

LAB02 · SEP 28-29, 2026

Schedule

1. What we want to see
2. Three ways to work with Aider
3. Two evenings, side by side
4. Hackathon 1.5

**What we want to
see**

What we already know



**A good model can one-shot
this**

Even Luna, on its own.



You used other tools

And other models.



**You focused too much on
output**

It is not what we are grading

WHAT WE WANT TO SEE

What we are looking for



Not the final product

Whether it works is the least interesting part.



Your design and your process

What you decided, how you asked, how you checked.

We are relaxed about grades. Trying something new counts.

Specification versus your own design

We specified

- A working Taskr and its data
- One permission model
- Outcomes, not implementations
- A test user to play

You designed

- The actions and what they return
- The loop and when it stops
- What the model sees
- What the user approves

Setup on the night



SSL certificates

Python could not verify the endpoint.



WSL and git hooks

Hooks that would not run.



Keys and environments

A key, a .venv, a first push.

Next time: setup released a week ahead, checked at home.

Three ways to work with Aider

HACKATHON 1, AND HOW WE PICTURED THE EVENING

Three ways to work with Aider

	You tell Aider	Who decides	Good for
Fixing	What is broken	Aider picks the fix	Getting unstuck fast
Designing	What the user needs	You	Starting each piece
Iterating	What you saw when you ran it	You	Finding what you did not know

All three are useful. Each has its own ways to go wrong.

Fixing: Aider picks the fix



When it works

Often. Aider reads the error, picks a reasonable fix, and you read what it changed.



Ways to get it wrong

The fix undoes something you decided
The same "fix it" again, with nothing new
A change committed that nobody read

Designing: you decide



When it works

"make sure ask mode actually prints the change BEFORE asking y/n, not after - that's what I'll be demoing by hand"

A decision about what the user sees, carried into the prompt.



Ways to get it wrong

Aider writes each section and you approve it

A design too long to hold in your head

Decisions that never reach a prompt

Iterating: you look

Time	What running it showed
8:10 PM	"Tomorrow" was September 26. It is the 25th.
8:10 PM	It looked frozen. It printed no prompt.
8:20 PM	Class at 1 to 4 PM. It is 9 to 12.
8:22 PM	It guessed about the demo instead of asking.

One student, playing the user after each piece. No test caught any of these.



Ways to get it wrong

A step too big to tell what changed
Running the tests, never the program
Asking Aider if it's done instead

The evening we pictured

7:00 • Read

The brief, as the user.
What is their Friday?
What would annoy
them?

[Designing](#)

7:15 • Decide

A short page of
decisions. Yours, stated
plainly.

[Designing](#)

7:30 • Repeat

Say what the next piece
is for. Aider builds it.
Run it. Say what you
saw. Fix what broke.

[All three](#)

8:40 • Play it

The whole scenario,
start to finish. Save it as
evidence.

[Iterating](#)

The loop uses all three. Most trouble came from leaving out the running.

Two evenings, side by side

THE THREE WAYS, AS TWO STUDENTS USED THEM

Two evenings

Student A

7:17 A one-page design, in their words

7:28 First piece: spec, build, run it

8:10 Running it finds four problems

8:33 The full scenario, saved as evidence

9:00 Working. 16 prompts.

Student B

7:33 Aider starts writing the design

8:14 27 commits and 2,900 words later, done

8:34 First code

8:48 First run: crash

9:00 Still crashing. 77 prompts.

Same brief, same model, same two hours. The next three slides take one way each.

Designing, side by side

	Student A: wrote it themselves	Student B: had Aider write it
How	One page in 17 minutes, before asking Aider anything	27 commits and 2,900 words in 41 minutes
What was in it	Their decisions. One: Taskr still says the assignment takes 3 hours, but it's 2, so ask	Detail B never checked. One: today's date is saved under "date". The file says "today"
Later	The assistant guessed instead of asking. A pointed Aider back at that decision, and it was fixed	That line became code and crashed the assistant at 8:48

A design helps you only if the decisions in it are yours.

Iterating, side by side



Student A ran it

After each piece, the assistant by hand, playing the user.

8:10 to 8:22: four problems found, none by a test.



Student B asked about it

Student: am I done implementing the spec?

Three times, 8:41 to 8:45. Each time Aider listed more to fix. First run: about 8:48.

A looked at the program. B asked Aider about the code.

Fixing, side by side



Student A: tests broke after a change

Student: 2 test bugs, not ai.py: ...

They said which side was wrong. Aider fixed the tests, and the assistant stayed as designed.



Student B: crash at startup

Student: fix it · is it fixed? · finish the rest of the fix · why is not working · why is not working · implement the fix

Aider changed the launcher three times. The bug was the "date" key. Still crashing at 9.

A's fixes had a decision behind them. B's had only the error.

What made the difference

B followed the process

Design first, then a spec, then check against it, then fix. Every step is on the brief's list.

Aider did every step

It wrote the design, judged "done", and chose the fixes. B approved.

A kept three things

The decisions, the looking, and what to tell Aider. Aider still wrote the code.

Each way works when the part that is yours stays yours.

Hackathon 1.5

A PROBLEM, NOT A SPEC

What Hackathon 1.5 is

- A lab assignment, started today
- Built on your Hackathon 1 assistant
- Same tools as the night
- Judged on design and process

This deck is the whole brief.

No starter code, no spec

Hackathon 1 gave you

- Outcomes A1 to A7
- A design template, a question per section
- A spec template per increment
- A scripted user

Hackathon 1.5 gives you

- One problem, on the next slide
- The user's files, to save and link

You write the spec.

Working out what the user means is the assignment.

Files that go with tasks

Today a task is a title, a due date and an estimate. What the work actually is lives in files the assistant never sees.

New

The user hands over files: an assignment handout, a syllabus, meeting notes, an email

The assistant saves each one and links it to its tasks

When it plans or answers, it goes and reads the right file

What the user wants

Hand over a file once, never explain it again

Ask about a task, get answers from its files

See which files go with which task, and fix a wrong link

Where files live, how they find their tasks, and when the assistant reads one are yours to decide. Every write still passes one check in your code.

Work on a new branch, not main: `git switch -c hackathon-1.5`

Questions?